CMIS 3153 · ADVANCED DATABASE SYSTEMS

Mid Exam Study Pack · Lesson 1

Data Storage and File Organization

Lesson in simple English · Formula cheat box · Answer recipes · Tutorial answers · Past paper questions 2020/21 – 2023/24 with answers

INSIDE THIS PACK

- How to use this pack
- Lesson part 1: Storage and the hard disk
- Lesson part 2: Access time, records and file organizations
- Cheat box: every formula for Lesson 1
- How-to-answer recipes
- Tutorial 1: Disk and file calculations (full answers)
- Tutorial 2 (Q1–Q12): Records and file organization
- Past paper questions with answers

★ asked in more than one past paper · (25m) marks in the paper · Yellow boxes = past paper questions · Blue boxes = formulas and key notes · Green boxes = given data

Contents

How to use this pack 4

Quick-find table 4

Lesson part 1: Storage and the hard disk 5

1.1 Types of storage 5

1.2 Disk vs main memory (both are "direct access") 5

1.3 Parts of a hard disk 6

Lesson part 2: Access time, records and file organizations 7

2.1 The three parts of disk access time 7

2.2 Disk factors that make queries slow (even with a fast CPU) 7

2.3 Buffering and double buffering 7

2.4 Records and blocking 8

2.5 Allocating file blocks on disk 8

2.6 File operations 9

2.7 The three main file organizations 9

2.8 Hashing in a little more detail 9

2.9 RAID 10

2.10 SAN and NAS 10

Cheat box: every formula for Lesson 1 11

Symbols 11

Disk timing 11

Disk capacity 12

File calculations 12

Search costs (in block accesses) 13

How-to-answer recipes 14

Recipe 1: Average time for one disk read ★ 14

Recipe 2: Record file calculation (R, bfr, b, search times) ★ 14

Recipe 3: Disk capacity and transfer rate 14

Recipe 4: Hash table with linear probing 14

Recipe 5: Theory questions ("Discuss", "Explain", "Compare") 14

Recipe 6: "Sequential vs indexed file for frequent updates" ★ 15

Tutorial 1: Disk and file calculations (full answers) 16

Question 1: Disk characteristics 16

Question 2: STUDENT file (= 2021/22 Q6) ★ 17

Questions 3 and 4: EMPLOYEE file 17

Tutorial 2 (Q1–Q12): Records and file organization 19

Q1 (and Q7). Reasons for variable-length records 19

Q2 (and Q8). How do we know the size of a variable-length record when storing it? 19

Q3. One table per file vs many tables in one file 19

Q4. Why does the allocation of records to blocks affect performance so much? 19

Q5. Logical files vs physical files 20

Q6. How does file size affect the choice of file organization? 20

Q9. When is it best to use a fixed-length format for a variable-length record? 20

Q10. What is stored in a file header? 20

Q11. File organization vs access method 21

Q12. Major problems with sorted files 21

Past paper questions with answers 22

A. Disk hardware 22

B. Disk access time and performance 23

C. Buffering and storing files 24

D. File calculation 25

E. Records and files 25

F. Sequential vs indexed organization 27

G. Hashing 27

H. RAID 28

Questions NOT in the mid scope 29

How to use this pack

Lesson 1 is almost always **Question 1** of the paper (since 2022/23), and it is the easiest place to collect marks because the calculations follow fixed formulas.

In the exam hall:

1. Read the question and find its type in the Quick-find table below.
2. Jump to that part. For theory, write the points in your own words. For calculations, copy the formula from the Cheat box, then put in the numbers.
3. Match the length to the marks: a **25-mark** part needs about **4 to 6 clear points** or one full calculation with every step shown.

Signs used in this pack: ★ = asked in more than one past paper. (25m) = marks given in the paper. AY = academic year of the paper.

Quick-find table

If the question asks about...	Go to
Memory hierarchy, primary vs secondary storage, disk vs main memory	Lesson part 1 · Past paper 2021/22 Q2(a)
Parts of a hard disk (platter, track, sector, cylinder, head)	Lesson part 1 · Past paper 2022/23 Q7(c)
Seek time, rotational delay, transfer time ★	Lesson part 2 · Cheat box · 2021/22 Q2(b), 2020/21 Q3(d)
Disk factors that slow down queries ★	Lesson part 2 · 2022/23 Q1(a), 2023/24 Q1(a)
Buffering, double buffering ★	Lesson part 2 · 2022/23 Q1(b), 2023/24 Q1(b)
Average disk access time calculation (7200 rpm) ★	Cheat box · Recipe 1 · 2022/23 Q1(c), 2023/24 Q1(c)
Record size, blocking factor, number of blocks, search time ★	Cheat box · Recipe 2 · Tutorial 1 Q2 = 2021/22 Q6
Track / cylinder / disk-pack capacity, transfer rate	Recipe 3 · Tutorial 1 Q1
Fixed vs variable length records, spanned vs unspanned	Lesson part 2 · Tutorial 2 · 2020/21 Q3(a)(b)
Block address, file header, file directory	Lesson part 2 · Tutorial 2 Q10 · 2020/21 Q3(c), 2022/23 Q7(d)
Heap vs sorted vs hash files, sequential vs indexed for frequent updates ★	Lesson part 2 · 2022/23 Q1(d), 2023/24 Q1(d)
Hashing, collisions, linear probing table	Lesson part 2 · Recipe 4 · 2020/21 Q5(c)
Storing a big file that is scanned sequentially	2021/22 Q2(c)
RAID	Lesson part 2 · 2021/22 Q2(d)

Lesson part 1: Storage and the hard disk

A database lives on disk, but the CPU can only work on data in main memory. So every read means: **find the block on disk → copy it to memory → process it → write it back if changed.** Disk is slow, so most of this lesson is about reducing disk work.

1.1 Types of storage

Type	Examples	Volatile? (lost when power off)	Speed / cost / size
Primary	Cache (static RAM), main memory (DRAM)	Yes	Fastest, most expensive, smallest. CPU works on it directly.
Secondary	Hard disk (HDD), SSD, flash, USB	No	Slower, cheaper, much bigger. Data must be copied to RAM first.
Tertiary	Magnetic tape, optical jukebox, cloud archives	No	Slowest, cheapest. Used for backup and archives (offline).
Cloud	Remote servers over the internet	No	Scalable, accessed from anywhere.

Memory hierarchy: put the **fast, small, expensive** memory close to the CPU and the **slow, big, cheap** storage further away. Order from top: registers → cache → main memory → flash/SSD → magnetic disk → optical disk → tape. This balances speed and cost.

Why databases are stored on disk (not in RAM):

- Databases are **too big** to fit in main memory.
- Disk is **non-volatile** (data survives power loss). RAM is volatile.
- Disk is about **10 times cheaper** per byte.
- Disk is **online** and can be accessed directly at any time (tape is offline).

1.2 Disk vs main memory (both are "direct access")

Point	Main memory	Disk
Speed	Nanoseconds	Milliseconds (thousands of times slower)
Why	Pure electronics	Moving parts: the arm must move (seek) and the platter must rotate
Unit of access	One byte or word	A whole **block** (512–8192 bytes)
Volatile	Yes	No
Cost per byte	High	Low
CPU access	Directly	Only after copying the block into memory

1.3 Parts of a hard disk

- **Platter (disk):** a round plate coated with magnetic material. **Single-sided** = data on one surface, **double-sided** = both surfaces.
- **Disk pack:** many platters stacked on one spindle.
- **Track:** one circle on a surface. A surface has hundreds to thousands of tracks.
- **Cylinder:** the tracks with the **same diameter on all surfaces**. Reading one cylinder needs no arm movement, so it is fast.
- **Sector:** a fixed arc of a track, set when the disk is formatted (usually 512 bytes). The smallest unit the hardware reads.
- **Block:** a group of sectors on one track. The **unit of transfer** between disk and memory. Size is fixed at formatting (512–8192 bytes).
- **Interblock gap:** a gap between blocks that holds control information, not data.
- **Read/write head:** reads or writes the bits. One head per surface, on an **arm**.
- **Actuator:** a motor that moves all arms together to the right cylinder.
- **Spindle motor:** spins the pack at a constant speed (5,400–15,000 rpm).
- **Disk controller:** hardware (with an interface such as SCSI) that takes the I/O command, moves the arm and does the read or write.

Block address (hardware address) = cylinder number + track (surface) number + block number within the track.

Fixed-head vs movable-head disks: a fixed-head disk has one head per track, so no arm movement is needed (fast but expensive). Normal disks are movable-head (one arm moves).

Lesson part 2: Access time, records and file organizations

2.1 The three parts of disk access time

To read one block, the disk spends time on three things. **Block access time = s + rd + btt.**

1. **Seek time (s):** time to move the arm and head to the correct **track/cylinder**. Usually 3–10 ms. The biggest part.
2. **Rotational delay or latency (rd):** time waiting for the disk to **spin the block under the head**. On average this is **half a rotation**.
3. **Block transfer time (btt):** time to actually **read the bytes** of the block as it passes under the head. Small.

Seek and rotation are much bigger than transfer. So the main trick is: **read many blocks that sit next to each other**, so you pay seek and rotation **once** only.

- k random blocks: k × (s + rd + btt)
- k consecutive blocks on one track/cylinder: s + rd + k × btt

Bulk transfer rate (btr): when reading consecutive blocks, the head also passes over the gaps, so the useful rate is a bit lower than tr: btr = (B / (B + G)) × tr.

2.2 Disk factors that make queries slow (even with a fast CPU)

- **Seek time:** data scattered over many cylinders means the arm keeps moving.
- **Rotational delay:** slow disks (low rpm) wait longer for each block.
- **Transfer rate:** a slow interface or small blocks limit throughput.
- **Random vs sequential access:** random reads pay seek + rotation for every block.
- **Fragmentation / poor placement:** a file spread over the disk turns a scan into many random reads.
- **Block size and blocking factor:** few records per block means more blocks to read.
- **No index:** a search must scan the whole file (b/2 blocks on average).
- **Small buffer pool:** the same blocks are read from disk again and again.
- **Contention:** many users or queries share one disk, so requests queue up.

2.3 Buffering and double buffering

Buffer = an area of main memory that holds disk blocks. A **buffer manager** keeps recently used blocks in memory so they do not have to be read again.

Double buffering: use two buffers, A and B. While the CPU **processes the block in buffer A**, the disk controller (an independent I/O processor) **reads the next block into buffer B**. Then they swap. Reading and processing happen **at the same time (in parallel)**.

Why it matters:

- The CPU does not sit idle waiting for the disk.
- Consecutive blocks stream in continuously, so **seek and rotational delay are paid only for the first block**.

- Fewer disk reads overall when blocks are reused from memory.
- It smooths the big speed gap between the fast CPU and slow disk.

2.4 Records and blocking

- **Record** = a collection of field values about one thing (for example one EMPLOYEE). **Record type** = field names + data types.
- **File** = a sequence of records stored in disk blocks.
- **Fixed-length records:** every record has the same size. Each field starts at a known byte position, so it is easy to find fields. May waste space.
- **Variable-length records:** records have different sizes. Reasons (Tutorial 2 Q1):
  1. **Variable-length fields** (for example a Name of any length).
  2. **Repeating fields** (a field with many values, such as several phone numbers).
  3. **Optional fields** (some records have no value).
  4. **Mixed file** (records of different types stored together, such as a STUDENT followed by its GRADE_REPORT records).
- To read variable-length records the system needs **separator characters** or **length fields** before each field.
- **Blocking factor (bfr)** = number of records that fit in one block = floor(B / R).
- **Unspanned:** a record must fit **inside one block**. Simple, each record starts at a known place. Wastes B − bfr × R bytes per block.
- **Spanned:** a record can **continue in the next block** using a pointer. No wasted space. **Needed when a record is bigger than a block (R > B).**

2.5 Allocating file blocks on disk

Method	How	Good	Bad
Contiguous	File blocks are next to each other	Very fast full read (with double buffering)	Hard to grow the file
Linked	Each block points to the next	Easy to grow	Slow to read the whole file
Clusters (extents)	Groups of consecutive blocks, groups linked	Mix of both	Some complexity
Indexed	Index blocks hold pointers to the file blocks	Direct access to any block	Extra index space

File header (file descriptor) holds: the **disk addresses of the file blocks**, and the **record format** (field lengths, field order, field type codes, separator characters, record type codes).

File directory: a system table that lists the files and, for each one, its name and where its header/blocks are, plus details such as size, owner, dates and access rights. The DBMS or operating system uses it to find a file's blocks quickly.

2.6 File operations

Open, Reset, Find (Locate), Read, FindNext, Delete, Modify, Insert, Close, FindOrdered, Reorganize.

- **File organization** = **how the data is arranged** on disk (records, blocks, access structures). Examples: heap, sorted, hash, B+ tree.
- **Access method** = **the group of operations** (programs) that you apply to a file. One organization can support several access methods, but some need a specific organization (no indexed access without an index).

2.7 The three main file organizations

	Heap (unordered)	Sorted (sequential, ordered)	Hash
How records are placed	At the end, any order	Ordered by an **ordering field**	Bucket chosen by **h(key)**
Insert	**Very fast** (add to last block)	**Slow** (find position, shift half the file)	Fast (one bucket)
Search by key	Linear: **b/2** blocks on average	Binary: **log2(b)** blocks	About **1** block access
Range search (<, >, BETWEEN)	Scan all b blocks	**Very good** (records are together)	**Not supported** (no order)
Delete	Leaves empty space; needs reorganizing	Use deletion markers + periodic reorganization	Fast
Main problem	Slow searches	Expensive inserts and updates	Collisions; no ranges; fixed size (static)

Problems with sorted files (Tutorial 2 Q12): inserting needs on average half the records moved; deleting leaves gaps; changing the ordering field moves the record; files usually need an **overflow file** and **periodic reorganization** to stay sorted.

2.8 Hashing in a little more detail

- A **hash function** h(K) turns a key into a **bucket number** (for example h(K) = K mod M). A bucket = one disk block or a cluster of blocks.
- **External hashing:** a table in the file header maps each bucket number to a disk block address.
- **Collision:** two keys hash to the same place. Fixes:
  - **Chaining:** keep a linked list (or overflow blocks) for each bucket.
  - **Open addressing:** look for the next free slot — **linear probing** (next slot, then next...), quadratic probing, or double hashing.
- **Static hashing problem:** the number of buckets is fixed, so the file cannot grow or shrink well.
- **Dynamic hashing schemes** let the file grow:
  - **Extendible hashing:** a **directory** of 2^d pointers (d = **global depth**); each bucket has a **local depth**. When a bucket overflows it splits, and the directory doubles only if needed.

    - **Linear hashing: no directory**; buckets are split one by one in a fixed (linear) order as the file grows.
- **Pros of hash indexes:** very fast equality search (about 1 I/O), fast insert and delete. **Cons:** useless for range queries and sorting, collisions slow it down, needs a good hash function, resizing (rehashing) is costly.

2.9 RAID

RAID = Redundant Array of Independent Disks. Many small disks act as **one logical disk**.

Objective: close the gap between fast CPUs/memory and slow disks (**performance**), and survive disk failures (**reliability**).

Key techniques:

- **Data striping:** split data into pieces and spread them over several disks, so they can be read **in parallel** (faster).
- **Mirroring:** keep an identical copy on another disk (safe).
- **Parity:** store extra check data so a lost disk can be **rebuilt**.

Level	Technique	Survives	Capacity	Best for
RAID 0	Striping only	0 disk failures	100%	Speed, temporary data
RAID 1	Mirroring	1 disk (per pair)	50%	Critical data, OS drives
RAID 5	Striping + distributed parity	1 disk	(n − 1) disks	File and web servers
RAID 6	Striping + double parity	2 disks	(n − 2) disks	Enterprise storage
RAID 10	Mirroring + striping	1 per mirrored pair	50%	Fast, critical databases

2.10 SAN and NAS

	SAN (Storage Area Network)	NAS (Network-Attached Storage)
Access level	**Block-level** (looks like a local disk to the server)	**File-level** (shared folders)
Network	Dedicated high-speed network (Fibre Channel, iSCSI)	Normal LAN (Ethernet), protocols like NFS/SMB
Used for	Databases, high-performance servers	File sharing, backups
Main idea	Storage separated from servers, pooled and managed centrally, scalable, redundant	A storage device with its own OS that serves files to many users

Cheat box: every formula for Lesson 1

Keep all times in **milliseconds (ms)** and all rates in **bytes per ms**. 1 second = 1000 ms. 1 minute = 60,000 ms. floor = round **down**, ceil = round **up**.

Symbols

Symbol	Meaning
p	Rotation speed in rpm (revolutions per minute)
s	Average seek time (ms)
rd	Average rotational delay (ms)
tr	Transfer rate (bytes/ms)
btr	Bulk transfer rate (bytes/ms)
btt	Block transfer time (ms)
B	Block size (bytes)
G	Interblock gap size (bytes)
R	Record size (bytes)
r	Number of records
bfr	Blocking factor (records per block)
b	Number of blocks in the file

Disk timing

Time for one full rotation:

$$T_{rev} = \frac{60{,}000}{p} \text{ ms}$$

Average rotational delay (half a rotation):

$$rd = \frac{1}{2} \times \frac{60{,}000}{p} = \frac{30{,}000}{p} \text{ ms}$$

Useful values: 2400 rpm → rotation 25 ms, rd 12.5 ms. 3600 rpm → 16.67 ms, rd 8.33 ms. 7200 rpm → 8.33 ms, rd 4.17 ms. 15,000 rpm → 4 ms, rd 2 ms.

Transfer rate (one whole track passes the head per rotation):

$$tr = \frac{\text{track size in bytes}}{T_{rev}} \text{ bytes/ms}$$

Block transfer time and bulk transfer rate:

$$btt = \frac{B}{tr} \qquad btr = \frac{B}{B+G} \times tr$$

Time to read one block (random block):

$$T_{block} = s + rd + btt$$

k random (scattered) blocks, and k consecutive blocks (simple and more accurate):

$$T_{random} = k \times (s + rd + btt)$$
$$T_{consecutive} = s + rd + k \times btt \approx s + rd + k \times \frac{B}{btr}$$

Transfer time when given a rate in MB/s and a data size (use 1 MB = 1000 KB unless told otherwise, and say which you used):

$$T_{transfer} = \frac{\text{data size}}{\text{transfer rate}}$$

Disk capacity

$$\text{surfaces} = \text{disks} \times 2 \ \ (\text{double-sided})$$

$$\text{track total} = \text{blocks per track} \times (B+G)$$
$$\text{track useful} = \text{blocks per track} \times B$$

$$\text{cylinder} = \text{surfaces} \times \text{track}$$
$$\text{number of cylinders} = \text{tracks per surface}$$
$$\text{disk pack} = \text{cylinders} \times \text{cylinder}$$

File calculations

$$R = \text{sum of all field sizes} + \text{deletion marker (if any)}$$

$$bfr = \left\lfloor \frac{B}{R} \right\rfloor$$
$$b = \left\lceil \frac{r}{bfr} \right\rceil$$
$$\text{wasted per block (unspanned)} = B - bfr \times R$$

Search costs (in block accesses)

Search	Average block accesses	Time
Linear search, record found	b / 2	Contiguous + double buffering: s + rd + (b/2) × btt (or × B/btr). Not contiguous: (b/2) × (s + rd + btt)
Linear search, not found	b	Same, with b instead of b/2
Binary search (sorted file)	ceil(log2 b)	ceil(log2 b) × (s + rd + btt), because every probe is a random block
Hash file, equality	about 1	s + rd + btt

Quick log2 values: log2(1000) ≈ 9.97, log2(1579) ≈ 10.62, log2(5000) ≈ 12.29, log2(10,000) ≈ 13.29, log2(100,000) ≈ 16.61.

How-to-answer recipes

Each recipe is a fixed set of steps. Follow them in order and show every line: examiners give marks for the formula and the method, not only the final number.

Recipe 1: Average time for one disk read ★

1. Write the formula: **T = s + rd + btt**.
2. Find **rd** from the rpm: rotation = 60,000 / p ms, rd = half of that.
3. Find **btt**: block size ÷ transfer rate (or B / tr). Watch the units: convert MB/s to bytes/ms or KB/ms.
4. Add them. Write the answer with **ms** and one line of meaning ("seek and rotation dominate").

Recipe 2: Record file calculation (R, bfr, b, search times) ★

1. **R** = add every field size + the deletion marker.
2. **bfr** = floor(B / R). Round **down** (a part record cannot fit when unspanned).
3. **b** = ceil(r / bfr). Round **up** (the last block may be part-full).
4. Linear search, average = **b/2** blocks.
   - Contiguous + double buffering: **s + rd + (b/2) × btt** (seek and rotation once).
   - Not contiguous: **(b/2) × (s + rd + btt)** (every block is random).
5. Binary search (file sorted on the key): **ceil(log2 b)** blocks × (s + rd + btt).
6. If a disk value is missing in the question (seek time or rpm), **write your assumption** clearly.

Recipe 3: Disk capacity and transfer rate

1. Surfaces = disks × 2 (if double-sided).
2. Track: total = blocks × (B + G), useful = blocks × B.
3. Cylinders = tracks per surface. Cylinder capacity = surfaces × track capacity.
4. Disk pack = cylinders × cylinder capacity.
5. tr = track total ÷ rotation time. btt = B / tr. btr = (B / (B + G)) × tr.

Recipe 4: Hash table with linear probing

1. Draw the empty table with slots 0 to (size − 1).
2. For each key in the given order: work out h(k) (for example k mod 10).
3. If that slot is empty, put the key there.
4. If it is taken (**collision**), try the next slot (+1), then the next, **wrapping from the last slot back to 0**, until an empty one is found.
5. Show a small working column (key → h(k) → slots tried → final slot), then the final table.

Recipe 5: Theory questions ("Discuss", "Explain", "Compare")

1. **Define** the term in one sentence.
2. Give **4–6 points** for a 25-mark part. For "compare", use a **two-column table**.

3. Add **one example** (for instance an employee table, a bank database).
4. End with **one line of conclusion** ("so indexed organization is better when updates are frequent").

Recipe 6: "Sequential vs indexed file for frequent updates" ★

Use this structure: define both → table of pros and cons → apply to the scenario (frequent inserts and updates) → conclusion: **indexed organization is better** for frequent updates and lookups, **sequential is only good** for batch processing of the whole file in order.

Tutorial 1: Disk and file calculations (full answers)

> **Exam link:** Question 2 of this tutorial came in the 2021/22 paper as Q6, **word for word (100 marks)**. Question 1 (e)–(f) is the same method as 2022/23 Q1(c) and 2023/24 Q1(c).

Question 1: Disk characteristics

> **Given:** block size B = 512 bytes, gap G = 128 bytes, 20 blocks per track, 400 tracks per surface, 15 **double-sided** disks.

First step: surfaces = 15 × 2 = **30 surfaces**.

(a) Total and useful capacity of a track

- Total = 20 × (B + G) = 20 × (512 + 128) = 20 × 640 = **12,800 bytes**
- Useful = 20 × B = 20 × 512 = **10,240 bytes** (gaps hold no user data)

(b) Number of cylinders

A cylinder = all tracks of the same diameter. Number of cylinders = tracks per surface = **400**.

(c) Capacity of a cylinder (one track from each of the 30 surfaces)

- Total = 30 × 12,800 = **384,000 bytes**
- Useful = 30 × 10,240 = **307,200 bytes**

(d) Capacity of the disk pack

- Total = 400 × 384,000 = **153,600,000 bytes** (about 153.6 MB)
- Useful = 400 × 307,200 = **122,880,000 bytes** (about 122.9 MB)

(e) Speed = 2400 rpm

1. One rotation = 60,000 / 2400 = **25 ms**
2. Transfer rate tr = track size / rotation time = 12,800 / 25 = **512 bytes/ms**
3. Block transfer time btt = B / tr = 512 / 512 = **1 ms**
4. Average rotational delay rd = half a rotation = 25 / 2 = **12.5 ms**
5. Bulk transfer rate btr = (B / (B + G)) × tr = (512 / 640) × 512 = **409.6 bytes/ms**

(f) Average seek time s = 30 ms. Time to locate and transfer one block

T = s + rd + btt = 30 + 12.5 + 1 = **43.5 ms**

(g) 20 random blocks vs 20 consecutive blocks with double buffering

- 20 random blocks: each pays everything → 20 × 43.5 = **870 ms**
- 20 consecutive blocks: seek and rotation once, then stream → s + rd + 20 × (B / btr) = 30 + 12.5 + 20 × (512 / 409.6) = 30 + 12.5 + 20 × 1.25 = **67.5 ms**
- (Simpler version with btt: 30 + 12.5 + 20 × 1 = 62.5 ms. Either is accepted if you show the method.)

- **Comparison:** consecutive reading is about **13 times faster** (870 / 67.5), because the seek time and rotational delay are paid only once.

Question 2: STUDENT file (= 2021/22 Q6) ★

> **Given:** r = 20,000 records. Fields: Name 30, Ssn 9, Address 40, PHONE 10, Birthdate 8, Sex 1, Major_dept_code 4, Minor_dept_code 4, Class_code 4, Degree_program 3, plus a 1-byte deletion marker. Same disk as Q1 (B = 512, s = 30 ms, rd = 12.5 ms, btt = 1 ms, btr = 409.6 bytes/ms).

(a) Record size R

R = 30 + 9 + 40 + 10 + 8 + 1 + 4 + 4 + 4 + 3 + 1 = **114 bytes**

(b) Blocking factor and number of blocks (unspanned)

- bfr = floor(512 / 114) = floor(4.49) = **4 records per block**
- b = ceil(20,000 / 4) = **5,000 blocks**

(c) Linear search, blocks contiguous, double buffering

- On average we read half the file: b / 2 = **2,500 blocks**.
- Contiguous + double buffering, so seek and rotation are paid once:
- T = s + rd + 2,500 × (B / btr) = 30 + 12.5 + 2,500 × 1.25 = **3,167.5 ms ≈ 3.17 seconds**
- (Simpler version with btt: 30 + 12.5 + 2,500 × 1 = 2,542.5 ms.)

(d) Linear search, blocks NOT contiguous

Every block is a random read: T = 2,500 × (s + rd + btt) = 2,500 × 43.5 = **108,750 ms ≈ 108.75 seconds**

(e) File ordered by Ssn, binary search

- Block accesses = ceil(log2 5,000) = ceil(12.29) = **13**
- Each one is a random read: T = 13 × 43.5 = **565.5 ms**

> **Note for the 2021/22 paper:** the paper does not give the rpm or seek time. Write: "Assuming 2400 rpm and an average seek time of 30 ms, as for this disk in Tutorial 1."

Questions 3 and 4: EMPLOYEE file

> **Given:** s = 20 ms, rd = 10 ms, btt = 1 ms, B = 2400 bytes, G = 600 bytes. Fields: Ssn 9, Last_name 20, First_name 20, Middle_init 1, Birth_date 10, Address 35, Phone 12, Supervisor_ssn 9, Department 4, Job_code 4, deletion marker 1. r = 30,000 records, fixed length, unspanned.

Record size, blocking factor, number of blocks

- R = 9 + 20 + 20 + 1 + 10 + 35 + 12 + 9 + 4 + 4 + 1 = **125 bytes**
- bfr = floor(2400 / 125) = floor(19.2) = **19 records per block**
- b = ceil(30,000 / 19) = ceil(1,578.9) = **1,579 blocks**

(a) Wasted space per block (unspanned)

Wasted = B − bfr × R = 2400 − 19 × 125 = 2400 − 2375 = **25 bytes per block**

(b) Transfer rate and bulk transfer rate

- tr = B / btt = 2400 / 1 = **2,400 bytes/ms**
- btr = (B / (B + G)) × tr = (2400 / 3000) × 2400 = **1,920 bytes/ms**

(c) Average block accesses for linear search

b / 2 = 1,579 / 2 = **789.5 ≈ 790 block accesses**

(d) Linear search time, consecutive blocks, double buffering

T = s + rd + 789.5 × (B / btr) = 20 + 10 + 789.5 × 1.25 = **≈ 1,016.9 ms (about 1.02 s)**. (With btt: 20 + 10 + 789.5 = 819.5 ms.)

(e) Linear search time, blocks NOT consecutive

T = 789.5 × (s + rd + btt) = 789.5 × 31 = **≈ 24,474.5 ms (about 24.5 s)**

(f) Binary search on the ordering key

- Block accesses = ceil(log2 1,579) = ceil(10.62) = **11**
- T = 11 × (20 + 10 + 1) = 11 × 31 = **341 ms**

> **What Tutorial 1 teaches:** reading blocks that sit next to each other (with double buffering) is many times faster than random reads, and a sorted file with binary search beats a linear scan. This is why indexes (Lesson 2) exist.

Tutorial 2 (Q1–Q12): Records and file organization

Q7 and Q8 are the same as Q1 and Q2, so they are answered once. Q13–Q15 (B-trees and B+ trees) are in the Lesson 2 pack.

Q1 (and Q7). Reasons for variable-length records

1. **Variable-length fields:** a field like Name uses only the characters it needs.
2. **Repeating fields:** one record has several values for a field (several phone numbers).
3. **Optional fields:** some records have no value for a field (for example Spouse_name).
4. **Mixed file:** records of **different types** are stored together, for example a STUDENT record followed by that student's GRADE_REPORT records in the same block.

Q2 (and Q8). How do we know the size of a variable-length record when storing it?

The system must know where each field and each record **ends**. Ways to do this:

- **Separator characters:** a special symbol (such as $ or %) after each field, and an end-of-record symbol.
- **Length fields:** store the length in bytes in front of each field, or the total record length at the start of the record.
- **Field name + value pairs** with a type code, useful for optional fields.
- **Offsets in a record header:** the header says where each variable field starts.

Record size = the actual field sizes + the separators or length fields. Each block then holds a different number of records, so bfr becomes an **average**, and **spanned** blocking can be used to avoid wasted space.

Q3. One table per file vs many tables in one file

Strategy	Advantage	Disadvantage
(a) Each table in its own file	Simple. Scanning one table reads only its own blocks. Easy to add or drop a table. Each table can use its own file organization.	A join must read several files, and related records are far apart, so more seeks. Many files to manage.
(b) Many tables (or the whole database) in one file	Related records can be **clustered** together (a department next to its employees), so joins need fewer block reads. Fewer files for the operating system.	Scanning **one** table must skip the other tables' records, so it is slower. Mixed record types make the file complex. Harder space management.

Q4. Why does the allocation of records to blocks affect performance so much?

- The **block** is the unit of disk I/O, so the cost of a query is roughly **the number of blocks read**.
- If records needed together are in the **same block or in consecutive blocks**, one read gives many useful records and seeks are avoided.

- If they are **scattered**, each record can cost a full random access (s + rd + btt, about 40 ms in Tutorial 1).
- Wasted space (unspanned blocks, deleted records) means more blocks for the same data, so more I/O.
- Example from Tutorial 1 Q2: linear search on contiguous blocks ≈ 3.2 s, on scattered blocks ≈ 109 s.

Q5. Logical files vs physical files

The lecture notes do not define these terms, so this answer uses the standard meanings.

- **Logical file:** the file **as programs and users see it**: a named collection of records that you Open, Find, Read and so on, without caring where it is stored. (In some systems a logical file is a view or index over a physical file.)
- **Physical file:** the **real data stored on the device**: blocks at actual disk addresses, with a file header that records the block addresses and record format.
- **Object file:** if it means a file of objects, it stores objects (their state and type) instead of flat records. (In programming, "object file" means compiled machine code, which is unrelated to databases.)
- **Differences:** logical = abstract, record view, used by programs. Physical = concrete, block view, on the disk. The DBMS or OS maps logical operations onto physical blocks, which gives **data independence** (programs do not change when storage changes).

Q6. How does file size affect the choice of file organization?

- **Small file** (a few blocks): a **heap** file is enough. A full scan is cheap, and extra structures are not worth the overhead.
- **Medium or large file with key lookups:** use a **sorted file with a primary index**, **hashing** (for exact-match searches) or a **B+ tree** (for exact match and ranges).
- **Very large and growing file:** static structures (sorted files, static hashing) slow down and need **reorganization**, so **dynamic** structures (B+ tree, extendible or linear hashing) are better.
- **Huge file read from start to end:** contiguous allocation, large blocks and double buffering.

Q9. When is it best to use a fixed-length format for a variable-length record?

When the lengths **vary only a little** (names are mostly under 30 characters), when there are **few optional fields**, or when a **repeating field has a small known maximum** (reserve 3 phone slots). You reserve the maximum size for each field (using null when empty). You gain fixed field positions, a simple bfr = floor(B / R), direct record addressing and easy in-place updates. You lose some space.

Q10. What is stored in a file header?

A **file header (file descriptor)** holds what programs need to use the file:

- **Disk addresses of the file blocks** (first block, last block for heap inserts, a block table, or the bucket-to-block table for hash files).
- **Record format:** for fixed-length records, the **field lengths and their order**; for variable-length records, **field type codes, separator characters and record type codes**.

- Often also: number of records, blocking factor, type of file organization and free-space information.

Q11. File organization vs access method

- **File organization** = **how the file's data is arranged** into records, blocks and access structures on the disk (heap, sorted, hash, B-tree).
- **Access method** = **the set of operations** a program can apply to the file (Find, FindNext, Insert, Delete...).
- One organization can support several access methods, but some access methods need a particular organization (you cannot use indexed access without an index).

Q12. Major problems with sorted files

1. **Insert is expensive:** find the right position, then **move on average half the records** (read and rewrite half the blocks).
2. **Delete leaves gaps** or needs shifting. Fixed with **deletion markers + periodic reorganization**.
3. **Changing the ordering field** moves the record (delete it, then insert it again).
4. Searching on **any other field** gets no benefit (still a linear search).
5. Even **binary search** costs log2(b) random block reads.
6. The usual fix is an **overflow (transaction) file** for new records that is merged in from time to time, but then every search must check both files.

Past paper questions with answers

Grouped by topic. Each question shows the paper year (AY = academic year) and marks. ★ = asked in more than one paper.

A. Disk hardware

> **2021/22 Q2(a) (25m).** Both disks and main memory support direct access to any location (page). Discuss the important differences between the two.

Both let you jump straight to any page (no need to read from the start, unlike tape). But:

Point	Main memory	Disk
Access time	Nanoseconds; same for every address	Milliseconds; depends on where the head is (seek + rotation)
How it works	Electronic, no moving parts	Mechanical: arm moves, platter spins
Unit of transfer	Byte or word	Whole block (page)
Volatility	Volatile (lost at power off)	Non-volatile (permanent)
Cost and size	Expensive, small	Cheap, very large
CPU use	CPU works on it directly	Block must be copied into memory first

So a DBMS tries to **reduce the number of disk block reads**: it places related data in the same or nearby blocks and keeps frequently used blocks in memory buffers.

> **2022/23 Q7(c) (25m, short note).** The mechanical components of hard disk drives, how they work and their importance.

- **Platters** coated with magnetic material, stacked on a **spindle**; a **spindle motor** spins them at a constant speed (5,400–15,000 rpm).
- Each surface has **tracks** (circles) divided into **sectors**; tracks of the same radius on all surfaces form a **cylinder**.
- **Read/write heads** (one per surface) sit on **arms**; an **actuator** moves all arms together to the required cylinder.
- **Disk controller** receives the block address (cylinder, surface, block), moves the arm, waits for rotation and transfers the data.
- **How a read works:** seek (arm moves) → rotational delay (block spins under the head) → transfer.
- **Importance:** these moving parts make disk access slow (milliseconds), so they decide database performance. This is why DBMSs use blocking, buffering, contiguous placement, indexes and RAID.

B. Disk access time and performance

> **2020/21 Q3(d) (25m) and 2021/22 Q2(b) (25m) ★** Discuss the time components involved in accessing a disk block. / Explain seek time, rotational delay and transfer time.

Block access time = **s + rd + btt**.

1. **Seek time (s):** time for the actuator to move the read/write head to the correct track (cylinder). Mechanical, depends on the distance moved. Typically 3–10 ms. Usually the largest part.
2. **Rotational delay (rd):** after reaching the track, the time waiting for the required block to rotate under the head. On average half a rotation: rd = (1/2) × (60,000 / p) ms. Example: 7200 rpm → 4.17 ms. Faster rotation = smaller delay.
3. **Block transfer time (btt):** time for the block's bytes to pass under the head: btt = B / tr. Much smaller than s and rd.

Key point: s and rd dominate, so reading **consecutive blocks** costs s + rd + k × btt, far less than k random blocks costing k × (s + rd + btt).

> **2022/23 Q1(a) (25m) and 2023/24 Q1(a) (25m) ★** Key factors that influence disk performance, with real-world examples. / How disk factors can cause slow query responses even when CPU and memory are sufficient.

The query is **I/O-bound**: the CPU sits idle waiting for the slow, mechanical disk (milliseconds per block, about a million times slower than memory).

Factor	Effect	Real-world example
Seek time	Arm moves for every random read	Banking or ticket-booking systems doing many random lookups slow down
Rotation speed (rpm)	Low rpm = longer wait and slower transfer	5,400 rpm disk (rd 5.6 ms) vs 15,000 rpm (rd 2 ms)
Transfer rate and block size	Small blocks = more requests; slow rate = slow scans	A report that scans a whole table
Data placement / fragmentation	Scattered blocks pay seek + rotation every time	Tutorial 1: contiguous scan 3.2 s vs scattered 109 s
Random vs sequential access	Random about 10–40 ms per block, sequential about 1 ms	Range query through a non-clustered index
Too little buffering / caching	Same blocks read from disk again and again	Buffer pool smaller than the data in use
Contention (many users)	Requests queue for one disk	Month-end batch jobs slow everyone
RAID level	Parity calculation slows writes	A write-heavy log on RAID 5

Remedies: proper (clustered) indexes, contiguous storage and clustering, larger blocks and prefetching, double buffering and caching, SSDs, RAID 0/10 striping, query optimization.

2022/23 Q1(c) (25m) ★ 7200 rpm, average seek time 8 ms, average latency 4.17 ms (from the rpm). Calculate the average time for a disk operation that involves both seeking and latency.

1. Check the latency: one rotation = 60,000 / 7200 = 8.33 ms, so average latency = 8.33 / 2 = **4.17 ms ✔**
2. Average time = seek + latency = 8 + 4.17 = **12.17 ms**
3. (If a transfer time were given, it would be added: s + rd + btt.)

2023/24 Q1(c) (25m) ★ 7200 rpm, average seek 9 ms, latency = half a rotation, transfer rate 120 MB/s, block size 64 KB. Command processing overlaps with seek time. Transfer starts only after seek and latency. Find the average time for one disk read.

1. Formula: **T = s + rd + btt**
2. Seek: s = **9 ms** (command processing overlaps with it, so nothing extra is added).
3. Rotational latency: one rotation = 60,000 / 7200 = 8.33 ms → rd = 8.33 / 2 = **4.17 ms**
4. Transfer: btt = 64 KB / 120 MB/s. Using 64 KB = 65,536 bytes and 120 MB/s = 120,000,000 bytes/s: 65,536 / 120,000,000 = 0.000546 s = **0.55 ms**. (If you use 1 MB = 1024 KB: 64 / (120 × 1024) s = 0.52 ms. State which you use.)
5. Total = 9 + 4.17 + 0.55 = **≈ 13.72 ms**

C. Buffering and storing files

2022/23 Q1(b) (25m) and 2023/24 Q1(b) (25m) ★ Define buffering in disk I/O and discuss its significance. / Why is buffering important for efficient interaction between the CPU and storage?

Buffering = keeping disk blocks in reserved areas of main memory (**buffers**) so the CPU works on data in memory while the disk transfers other blocks.

- **Double buffering:** while the CPU processes the block in buffer A, the disk controller (which works independently) fills buffer B with the next block; then they swap. Processing and reading **overlap** in parallel.
- **Hides disk delay:** the CPU does not wait idle for each slow disk read.
- **Continuous streaming:** consecutive blocks flow in without stopping, so seek and rotational delay are paid **only for the first block**.
- **Caching / reuse:** blocks already in the buffer pool are not read again, so there are fewer disk I/Os.
- **Write buffering:** changes are collected and written in batches.
- **Matches speeds:** smooths the large speed gap between the fast CPU and the slow disk.

- Example: a full table scan with double buffering reads 2,500 blocks in about 3.2 s instead of about 109 s with random reads (Tutorial 1).

> **2021/22 Q2(c) (25m).** If you have a large file that is frequently scanned sequentially, how would you store its pages on disk?

- Store the pages **contiguously**: consecutive blocks on the **same track**, then the **same cylinder**, then **neighbouring cylinders**. Then the head never makes long seeks.
- Use **contiguous allocation** or large **extents (clusters)** instead of linked or scattered blocks.
- Read with **double buffering / prefetching** (read-ahead of the next blocks) so seek and rotation are paid once.
- Use a **large block size** and a high blocking factor so fewer blocks are needed.
- Avoid **fragmentation**; reorganize the file if it becomes scattered.
- Result: time ≈ s + rd + k × btt instead of k × (s + rd + btt).

D. File calculation

> **2021/22 Q6 (100m) ★** The STUDENT file question (R, bfr, b, linear search contiguous, linear search non-contiguous, binary search). This is **exactly Tutorial 1 Q2**: see the full answer in the Tutorial 1 section above. Answers: R = 114 bytes, bfr = 4, b = 5,000 blocks, contiguous linear search ≈ 3,167.5 ms, non-contiguous ≈ 108,750 ms, binary search = 13 blocks ≈ 565.5 ms.

E. Records and files

> **2020/21 Q3(a) (25m).** Differentiate between fixed-length records and variable-length records in a file.

Fixed-length records	Variable-length records
Every record has the **same size** in bytes	Records have **different sizes**
Each field starts at a **known position**, so fields are found directly	Need **separator characters or length fields** to find where fields end
Easy to compute bfr = floor(B / R) and the address of the i-th record	bfr is only an average; record addresses must be stored
Easy to update in place	An update that makes a record longer may have to move it
May **waste space** (short values padded to the maximum)	**Saves space**
Example: Ssn char(9)	Example: Name of any length, optional or repeating fields, mixed record types

2020/21 Q3(b) (25m). What is spanned and unspanned organization of records?

- **Unspanned:** a record is **not allowed to cross a block boundary**; it must fit entirely in one block. Each record starts at a known place, so processing is simple. Unused space per block = B − bfr × R. Used for fixed-length records when B > R.
- **Spanned:** part of a record can be stored in one block and the **rest in another block**, linked by a **pointer** at the end of the first block. No wasted space. **Must be used when a record is larger than a block (R > B)**, and is useful for large variable-length records.
- Example: B = 512, R = 114 → unspanned holds 4 records and wastes 56 bytes per block; spanned uses those 56 bytes for part of the next record.

2020/21 Q3(c) (25m). What are the components of a disk block address?

The hardware (physical) address of a block has three parts:

1. **Cylinder number:** which cylinder (the set of tracks with the same radius on all surfaces), so the arm knows where to move.
2. **Track (surface) number:** which surface within that cylinder, so the right read/write head is chosen.
3. **Block number:** which block within that track, so the controller knows when it rotates under the head.

The address is given to the disk controller, which positions the head (seek), waits for rotation and transfers the block. (Some systems also use a **logical block address**, a single block number that the controller maps to these three parts.)

2022/23 Q7(d) (25m, short note). The role of file directories in file organization.

- A **file directory** is a system table (kept by the OS or the DBMS catalog) that lists every file and **where to find it**.
- For each file it stores the **name**, the location of its **file header** or first block (and so all its block addresses), size, number of records, owner, creation and change dates, and **access rights**.
- **Role:** to locate a file's blocks quickly when the file is opened; to organize files in a hierarchy (folders); to manage free space and allocation; to enforce security and sharing; to support file operations (Open, Close, Delete).
- Together with the file header (which describes record format and block addresses), the directory lets the system go from a file name to the exact disk blocks.

F. Sequential vs indexed organization

> **2022/23 Q1(d) (25m) and 2023/24 Q1(d) (25m) ★** Employee records (Employee ID, Name, Department, Salary) with frequent updates (promotions, department changes). Discuss the pros and cons of sequential file organization vs indexed file organization. / The suitability of sequential vs indexed organization for frequently updated records.

Sequential (sorted) file: records are stored in order of a key (for example Employee ID).

- Pros: very fast to read all records in order (payroll, reports); binary search on the key (log2 b blocks); simple, no extra space.
- Cons: **insert and delete are expensive** (shift half the file or use an overflow file); **changing the key moves the record**; searching on other fields (Department) needs a full scan; needs **periodic reorganization**.

Indexed file: the data file plus an **index** (for example a B+ tree on Employee ID, and a secondary index on Department).

- Pros: **fast direct access** to any employee (a few block reads); **inserts and updates are cheap** because the dynamic index (B+ tree) adjusts itself; can have several indexes for several search fields; supports both exact-match and range queries.
- Cons: **extra storage** for the index; every insert, delete or key update must **also update the index** (some overhead); more complex.

> **Conclusion for this scenario:** the records are updated often and looked up individually, so **indexed organization is the better choice**. Sequential organization only suits batch jobs that process the whole file in order and rarely change it.

G. Hashing

> **2020/21 Q5(c) (40m).** Insert 12, 18, 13, 2, 3, 23, 5, 15 into an empty hash table of length 10 using open addressing, h(k) = k mod 10 and linear probing. Draw the table.

Linear probing: try h(k); if full, try h(k)+1, h(k)+2, ... (wrap around after slot 9).

Key	h(k) = k mod 10	Slots tried	Stored in
12	2	2	**2**
18	8	8	**8**
13	3	3	**3**
2	2	2 (full) → 3 (full) → 4	**4**
3	3	3 → 4 → 5	**5**
23	3	3 → 4 → 5 → 6	**6**

Key	h(k) = k mod 10	Slots tried	Stored in
5	5	5 → 6 → 7	**7**
15	5	5 → 6 → 7 → 8 → 9	**9**

Final hash table:

Slot	0	1	2	3	4	5	6	7	8	9
Key	–	–	12	13	2	3	23	5	18	15

Extra comment: the collisions formed one long run of full slots (2 to 9), called **primary clustering**, so later keys needed many probes (15 needed 5). Quadratic probing or double hashing reduce this.

> **2021/22 Q1(c) (25m).** Discuss the pros and cons of using hash indexes in a database.

A **hash index** applies a hash function h(K) to the search key to find the **bucket** (one or more blocks) that holds the entry.

Pros:

- **Fastest exact-match search**: usually **one bucket access** (O(1)), fewer I/Os than a B+ tree.
- Fast insert and delete.
- Simple; great for equality joins and unique keys (session ID, token, customer ID).
- Dynamic hashing (extendible, linear) lets the file grow without a full rehash.

Cons:

- **No range queries** (<, >, BETWEEN), no ORDER BY, no prefix search (LIKE 'ab%'), because keys are not stored in order.
- **Collisions** cause overflow chains or probing; skewed data or a poor hash function make it slow.
- **Static hashing** has a fixed number of buckets; growth causes overflow and **rehashing is expensive**.
- Depends on an even spread of keys; a big bucket needs many comparisons.

> **Conclusion:** use a hash index for columns searched **only by exact match**; use a B+ tree when ranges or ordering are needed.

H. RAID

> **2021/22 Q2(d) (25m).** Discuss RAID architecture: objective, key techniques and a brief description of the RAID types.

RAID (Redundant Array of Independent Disks) combines many physical disks into **one logical disk**.

Objective: disks improve much more slowly than CPUs and memory. RAID uses many disks in parallel to **increase performance (speed)** and stores redundant data to **increase reliability** (survive

disk failures).

Key techniques:

- **Data striping:** split data into blocks or bits and spread them over several disks, so they are read and written **in parallel**.
- **Mirroring (shadowing):** write identical copies to two disks; if one fails the other is used.
- **Parity / error-correcting codes:** store check information so data on a failed disk can be **rebuilt**.

Types (levels):

- **RAID 0:** striping only. Fastest, full capacity, **no fault tolerance** (one failure loses all data). Used for temporary or video-editing data.
- **RAID 1:** mirroring. Survives one disk failure, fast reads, but only **50% capacity**. Used for critical databases and OS disks.
- **RAID 5:** striping with **distributed parity**. Survives **one** failure, capacity = (n − 1) disks, good reads, slower writes (parity). Used for file and web servers.
- **RAID 6:** striping with **double parity**. Survives **two** failures, capacity = (n − 2) disks, slower writes. Used for enterprise storage.
- **RAID 10 (1 + 0):** mirrored pairs that are then striped. Fast and very safe, but 50% capacity. Used for high-performance critical databases.

Questions NOT in the mid scope

Past paper questions on Object-Relational databases, Data Warehousing/OLAP, the three-schema architecture and NoSQL are **left out**, because the lecturer said the mid covers Lessons 1–5 only.